

P3357R0

NRVO with factory and after_factory

New Proposal, 2024-07-15

This version:

<http://virjacode.com/papers/p3357r0.htm>

Latest version:

<http://virjacode.com/papers/p3357.htm>

Author:

Thomas PK Healy <healytpk@vir7ja7code7.com> (*Remove all sevens from email address*)

Audience:

SG18

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Add two new functions to the standard library called `std::factory` and `std::after_factory` to facilitate the construction and modification of *prvalues*, particularly for objects returned by value from a function, achieving *Named Return Value Optimisation (NRVO)*, addressing a significant gap in core language functionality.

Table of Contents

1 Introduction

2 Motivations

2.1 Resource Management

2.2 Concurrency Control

3 Design Considerations

3.1 Usage

3.2 factory

3.3 after_factory

4 Possible Implementation

5 Impact on the Standard

6 Impact on Existing Code

7 Acknowledgements

References

Informative References

§ 1. Introduction

C++23 lacks *Named Return Value Optimization (NRVO)*. This limitation is often encountered in scenarios where objects need to be constructed, modified, and returned from a function, such as when working with synchronization primitives or other resource management types. This is particularly a problem if the return type is both unmovable and uncopyable, or if the return type is large and risks overflowing the stack.

This paper proposes a new feature which is inferior to the paper [\[P2025R2\]](#) authored by Anton Zhilin entitled "*Guaranteed copy elision for return variables*". Anton's paper proposes a superior solution to NRVO than what is proposed in this paper, however I foresee that Anton's paper will not make it in time into C++26, and perhaps not C++29 either. The intention of this proposal is that `std::factory` and `std::after_factory` shall be used until such time as Anton's paper is assimilated into the standard.

§ 2. Motivations

The proposed `std::factory` function provides a convenient and efficient way to work with unmovable-and-uncopyable return types, addressing a limitation in the language's handling of such types. By allowing the construction and immediate modification of objects before they are returned, `std::factory` enables the programmer to write cleaner and more expressive code. Furthermore, when dealing with types that are either movable or copyable, the use of `std::factory` will be more efficient as it avoids a copy/move operation.

Similarly, `std::after_factory` enables efficient post-construction modifications of the returned object. This functionality is particularly valuable for fine-tuning the object's state before finalising its return.

§ 2.1. Resource Management

`std::factory` simplifies resource management by encapsulating object creation and initialization within a single function call. This is especially useful for managing scarce or expensive resources such as file handles or database connections.

`std::after_factory` facilitates precise resource management by allowing modifications to the object after initial construction. This capability ensures that resources are acquired and initialized correctly before the function returns, thereby enhancing resource utilization efficiency.

§ 2.2. Concurrency Control

In multi-threaded applications, `std::factory` can streamline the initialization of synchronization primitives like mutexes or semaphores, ensuring proper locking semantics without the need for manual intervention.

`std::after_factory` complements this by enabling precise post-construction modifications to the returned object. In concurrent environments, this functionality ensures that any necessary adjustments to the object's state or synchronization setup can be safely applied before finalizing its return. This capability is essential for maintaining thread safety and ensuring correct behavior in complex, multithreaded scenarios.

§ 3. Design Considerations

The ability to specify constructor arguments directly within `std::factory` allows for more flexible object initialization, accommodating varying initialization requirements without cluttering the calling code.

Similarly, `std::after_factory` provides a structured approach to modifying the returned object's state post-construction. This feature enhances code clarity and maintainability by separating object creation from subsequent modifications, thereby facilitating more modular and understandable codebases.

§ 3.1. Usage

§ 3.2. factory

```
template<typename T, typename... Params, typename Setup>
T std::factory(Params&&... args, Setup &&setup);
```

`T` is the type of the object to be constructed

`args...` are the constructor arguments for `T`

`setup` is a callable object that takes a reference to the constructed object and modifies it.

Sample usage:

```
int main(void)
{
    int i = std::factory<int>(52, [](auto &a){ ++a;});
}
```

§ 3.3. after_factory

```
template<typename F, typename... Params, typename Setup>
invoke_result_t<F&&,Params&&> after_factory(F &&f, Params&&... args, Setup
```

`f` is the function or functor that returns by value

`args...` are the arguments to be passed to `f`

`setup` is a callable object that takes a reference to the constructed object and modifies it

Sample usage:

```
binary_semaphore Func(void)
{
    return std::factory<binary_semaphore>(0, [](auto &a){a.release();});
}

int main(void)
```

```

{
    binary_semaphore bs = std::after_factory(Func, [] (auto &a) { a.acquire();
    bs.release();
    bs.acquire();
}

```

§ 4. Possible Implementation

```

#include <cstdint>          // size_t
#include <bit>              // bit_cast
#include <functional>       // invoke
#include <tuple>            // apply, get, make_tuple, tuple, tuple_size
#include <type_traits>      // false_type, is_reference, is_rvalue_reference, ...
#include <utility>          // forward, index_sequence, make_index_sequence, ...

namespace std {

// ===== The following 6 lines are the implementation of is_specialization
template<typename Test, template<typename...> class Ref>
struct is_specialization : false_type {};
template<template<typename...> class Ref, typename... Args>
struct is_specialization<Ref<Args...>, Ref> : true_type {};
template<typename Test, template<typename...> class Ref>
inline constexpr bool is_specialization_v = is_specialization<Test,Ref>::value;
// =====

namespace detail {

// The following function prepends an index to a sequence
// Example Input: 5, index_sequence<0,1,2,3,4>
// Example Output: index_sequence<5,0,1,2,3,4>
template <size_t n, size_t... Ns>
constexpr auto PrependSeq(index_sequence<Ns...>)
{
    return index_sequence<n,Ns...>{};
}

// The following template type is an index_sequence
// but with the last element moved to the beginning

```

```

template<size_t n>
using Sequence = decltype(PrependSeq<n-1u>(make_index_sequence<n-1u>{}));

// Helper function to re-order the tuple so that
// the last element is moved to the beginning
template<typename Tuple, size_t... Indices>
requires is_rvalue_reference_v<Tuple&&> && is_specialization_v<Tuple, tuple>
constexpr auto ReorderTuple_impl(Tuple &&t, index_sequence<Indices...>)
{
    // Ensure that the tuple is just a tuple full of references
    static_assert( (is_reference_v<decltype(get<Indices>(move(t)))> && ...

    // Some references may be Lvalue and some may be Rvalue,
    // so we use decltype here to keep the correct type
    return tuple< decltype(get<Indices>(move(t)))... >{
        get<Indices>(move(t))...
    };
}

// Main function to re-order the tuple so that
// the last element is moved to the beginning
template<typename Tuple>
requires is_rvalue_reference_v<Tuple&&> && is_specialization_v<Tuple, tuple>
constexpr auto ReorderTuple(Tuple &&t)
{
    return ReorderTuple_impl(
        move(t),
        Sequence< tuple_size_v<Tuple> >{});
}

template<typename T> requires (!is_reference_v<T>)
struct Derived final {
    T var;
    template<typename Setup, typename... Params>
    constexpr Derived(Setup &&setup, Params&&... args)
        noexcept( noexcept(T{forward<Params>(args)...}) ) && noexcept(forward
        : var{ forward<Params>(args)... }
    {
        static_assert( sizeof (Derived) == sizeof (T) );
        static_assert( alignof(Derived) == alignof(T) );
        forward<Setup>(setup)( this->var );
    }
    template<typename Setup>

```

```

struct Maker final {
    template<typename... Params>
    static constexpr Derived Make(Setup &&setup, Params&&... args)
        noexcept(noexcept(Derived{static_cast<Setup&&>(setup), forward<Params>(args)...}))
    {
        // Note: 'setup' is not a forwarding ref so don't use 'forward'
        return Derived{ static_cast<Setup&&>(setup), forward<Params>(args)... };
    }
};

template<typename T, typename Setup, typename... Params> requires (!is_reference_v<T>)
T factory_impl(Setup &&setup, Params&&... args)
noexcept(noexcept(Derived<T>::template Maker<Setup&&>::template Make<Params&&...>(setup, args...)))
{
    // Cannot be constexpr because of the casts 10 lines further down
    constexpr bool noexcept_specs = noexcept(Derived<T>::template Maker<Setup&&>::template Make<Params&&...>(setup, args...));

    using SrcType = Derived<T> (*const)(Setup&&, Params&&...) noexcept(noexcept_specs);
    using DstType = T (*const)(Setup&&, Params&&...) noexcept(noexcept_specs);

    constexpr auto src = &Derived<T>::template Maker<Setup&&>::template Make<Params&&...>(setup, args...);
    static_assert( is_same_v<SrcType, decltype(src)> ); // not needed but

#ifdef 0
    DstType const dst = *static_cast<DstType const*>(static_cast<void const*>(src));
#else
    DstType const dst = bit_cast<DstType>(src);
#endif

    return dst( forward<Setup>(setup), forward<Params>(args)... );
}

template<typename T> requires (!is_reference_v<T>)
struct factory_helper {
    template<typename... Params>
    auto operator()(Params&&... args)
        noexcept(noexcept(factory_impl<T>(forward<Params>(args)...)))
    {
        // Cannot be constexpr because factory_impl is not constexpr
        return factory_impl<T>(forward<Params>(args)...);
    }
};

```

```

template<typename F, typename Setup, typename... Params>
auto after_factory_impl(F &&f, Setup &&setup, Params&&... args)
noexcept( noexcept( invoke( forward<F>(f), forward<Params>(args)... ) ) && noexcept )
{
    // Cannot be constexpr because of the cast 22 lines further down

    typedef invoke_result_t< F&&, Params&&... > R;
    static_assert( !is_reference_v<R>, "Callable must return by value" );

    struct S {
        R var;
        S(Setup &&setup, F &&f, Params&&... args)
            : var( invoke( forward<F>(f), forward<Params>(args)... ) )
        {
            static_assert( sizeof (S) == sizeof (R) );
            static_assert( alignof(S) == alignof(R) );
            forward<Setup>(setup)(this->var);
        }
    };

    constexpr auto x =
        +[](Setup &&s2, F &&f2, Params&&... args2) -> S
        {
            return S( forward<Setup>(s2), forward<F>(f2), forward<Params>(args2)... );
        };

    auto const x2 = (R(*) (Setup&&, F&&, Params&&... ))x;

    return x2(
        forward<Setup>(setup),
        forward<F>(f),
        forward<Params>(args)... );
}

template<typename F, typename Tuple, size_t... Indices>
requires is_rvalue_reference_v<Tuple&&> && is_specialization_v<Tuple, tuple>
auto after_factory_helper(F &&f, Tuple &&tup, index_sequence<Indices...>)
noexcept( noexcept( after_factory_impl( forward<F>(f), get<Indices>(move(tup))... ) ) )
{
    // Cannot be constexpr because 'after_factory_impl' isn't constexpr

    // Make sure the tuple is full of references:

```

```

        static_assert( (is_reference_v<decltype(get<Indices>(move(tup)))> && ...
        return after_factory_impl( forward<F>(f), get<Indices>(move(tup))... );
    }

} // close namespace 'detail'

template<typename T, typename... ParamsAndSetup>
T factory(ParamsAndSetup&&... pands)
noexcept(noexcept(apply(detail::factory_helper<T>{}, detail::ReorderTuple(tup),
{
    // Cannot be constexpr because 'factory_helper::operator()' isn't constexpr
    using detail::factory_helper, detail::ReorderTuple;
    return apply(
        factory_helper<T>{},
        ReorderTuple( tuple<ParamsAndSetup&&...>{ static_cast<ParamsAndSetup>(pands)... } )
    );
}

template<typename F, typename... ParamsAndSetup>
auto after_factory(F &&f, ParamsAndSetup&&... pands)
noexcept(noexcept(detail::after_factory_helper(std::forward<F>(f), detail::ReorderTuple(tup),
{
    // Cannot be constexpr because 'after_factory_helper' isn't constexpr
    using detail::after_factory_helper, detail::ReorderTuple;
    return after_factory_helper(
        forward<F>(f),
        ReorderTuple( tuple<ParamsAndSetup&&...>{ forward<ParamsAndSetup>(pands)... } )
        make_index_sequence< sizeof...(ParamsAndSetup) >{} );
}

} // close namespace 'std'

```

Tested and working up on GodBolt: <https://godbolt.org/z/doKr5fq8T>

Note that compiler support is required to make these two library functions `constexpr`.

§ 5. Impact on the Standard

The additions of `std::factory` and `std::after_factory` constitute a library extension that has no impact on existing language features or behavior.

§ 6. Impact on Existing Code

No existing code becomes ill-formed due to the introduction of `std::factory` and `std::after_factory`. The behavior of existing code is unaffected.

§ 7. Acknowledgements

This was Sebastian Wittmeier's idea.

§ References

§ Informative References

[P2025R2]

Anton Zhilin. *Guaranteed copy elision for return variables*. 15 March 2021. URL: <https://wg21.link/p2025r2>